EX. NO: 1 IMPLEMENTATION OF DDL COMMANDS OF SQL WITH SUITABLE EXAMPLES

(DATA DEFINITION LANGUAGE COMMANDS)

AIM

To create a database to perform creation of table, alter, modify and drop column.

CREATE COMMAND:

The create command defines each column of the table uniquely. Each column has minimum of three attributes, a name, data type and size.

Syntax:

create table <table_name>(column_name datatype(size));

ALTER COMMAND:

a) The alter command when used with add allow us to add additional column to an already existing table.

Syntax:

alter table <table_name> add column_name datatype (size);

b) The alter command when used with modify expression redefines the column with the given values but cannot change the column names.

Syntax:

alter table <table_name> modify column_name datatype (size);

c) The alter command when used with drop deletes the specified column in the table

Syntax:

alter table <table name> drop column_name;.

DROP COMMAND:

A table can be dropped (deleted) by using drop command.

Syntax:

drop table <table_name>;

TRUNCATE COMMAND:

This command is used to delete all the entries to a table but however the table structure remains unchanged.

Syntax:

truncate table <table_name>;

DESCRIBE COMMAND:

This command is used to display the structure of a table along with fields and constraints.

Syntax:

desc <table_name>;

COMMANDS

```
mysql> create database a;
Query OK, 1 row affected (0.00 sec)
```

```
mysql> use a;
Database changed
mysql> create table stud (sname varchar(30), sid varchar(10), sage integer, sarea varchar(20));
Query OK, 0 rows affected (0.09 sec)

mysql> show tables;
+---------------+
| Tables_in_a |
+---------------+
| stud        |
+---------------+
1 row in set (0.00 sec)

mysql> desc  stud;
+--------+----------------+--------+------+-----------+--------+
| Field | Type         | Null | Key | Default | Extra |
+--------+----------------+--------+------+-----------+--------+
| sname | varchar(30) | YES  |     | NULL    |       |
| sid   | varchar(10) | YES  |     | NULL    |       |
| sage  | int(11)     | YES  |     | NULL    |       |
| sarea | varchar(20) | YES  |     | NULL    |       |
+--------+----------------+--------+------+-----------+--------+
4 rows in set (0.00 sec)

mysql> alter table stud add phoneno integer after sage;
Query OK, 0 rows affected (0.23 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> desc stud;
+-----------+----------------+--------+------+-----------+--------+
| Field    | Type         | Null | Key | Default | Extra |
+-----------+----------------+--------+------+-----------+--------+
| sname    | varchar(30) | YES  |     | NULL    |       |
| sid      | varchar(10) | YES  |     | NULL    |       |
| sage     | int(11)     | YES  |     | NULL    |       |
| phonenum | int(11)     | YES  |     | NULL    |       |
| sarea    | varchar(20) | YES  |     | NULL    |       |
+-----------+----------------+--------+------+-----------+--------+
5 rows in set (0.00 sec)


mysql> alter table stud modify sage varchar(20);
Query OK, 0 rows affected (0.22 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> desc stud;
```

```
+------------+----------------+--------+------+-----------+---------+
| Field    | Type         | Null | Key | Default | Extra |
+------------+----------------+--------+------+-----------+---------+
| sname    | varchar(30) | YES  |     | NULL    |       |
| sid      | varchar(10) | YES  |     | NULL    |       |
| sage     | varchar(20) | YES  |     | NULL    |       |
| phonenum | int(11)     | YES  |     | NULL    |       |
| sarea    | varchar(20) | YES  |     | NULL    |       |
+------------+----------------+--------+------+-----------+---------+
5 rows in set (0.00 sec)


mysql> alter table stud drop sage;
Query OK, 0 rows affected (0.39 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> desc stud;
+------------+----------------+--------+------+-----------+---------+
| Field    | Type         | Null | Key | Default | Extra |
+------------+----------------+--------+------+-----------+---------+
| sname    | varchar(30) | YES  |     | NULL    |       |
| sid      | varchar(10) | YES  |     | NULL    |       |
| phonenum | int(11)     | YES  |     | NULL    |       |
| sarea    | varchar(20) | YES  |     | NULL    |       |
+------------+----------------+--------+------+-----------+---------+
4 rows in set (0.00 sec)

mysql> truncate table stud;
Query OK, 0 rows affected (0.05 sec)

mysql> desc stud;
+------------+----------------+--------+------+-----------+---------+
| Field    | Type         | Null | Key | Default | Extra |
+------------+----------------+--------+------+-----------+---------+
| sname    | varchar(30) | YES  |     | NULL    |       |
| sid      | varchar(10) | YES  |     | NULL    |       |
| phonenum | int(11)     | YES  |     | NULL    |       |
| sarea    | varchar(20) | YES  |     | NULL    |       |
+------------+----------------+--------+------+-----------+---------+
4 rows in set (0.00 sec)

mysql> drop table stud;
Query OK, 0 rows affected (0.05 sec)

mysql> desc stud;
ERROR 1146 (42S02): Table 'a.stud' doesn't exist
```

RESULT

Thus the data definition commands have been executed successfully

EX. NO: 2 IMPLEMENTATION OF DML COMMANDS OF SQL WITH SUITABLE EXAMPLES

(DATA MANIPULATION LANGUAGE COMMANDS)

AIM

To implement DML commands for inserting, deleting, updating and retrieving tables.

INSERT COMMAND:

This command is used to insert values into the table.

Syntax:

insert into <table_ name> (column_name) values(attribute_value);

DELETE COMMAND:

This command is used to delete rows from the table based on condition.

Syntax:

delete from <table_name>where condition;

ALTER COMMAND:

a) The alter command when used with add allow us to add additional column to an already existing table.

Syntax:

alter table <table_name>add column_name datatype (size);

b) The alter command when used with modify expression redefines the column with the given values but cannot change the column names.

Syntax:

alter table <table_name>modify column_name datatype (size);

c) The alter command when used with drop deletes the specified column in the table.

Syntax:

alter table <table_name>drop column_name;

UPDATE COMMAND:

A row in a table can be updated using update command.

Syntax:

update <table_name> set column_name=value where condition;

SELECT COMMAND:

This command is used to display the rows in the table based on condition

Syntax:

select column_name1,column_name2.....column_namen from <table_name> where condition ;

DELETE COMMAND:

This command is used to delete rows in the table based on condition

Syntax:

delete from <table_name> where condition ;

COMMANDS

CREATION OF TABLE

mysql> create table student(sn char(20),regno integer,address varchar(40));
Query OK, 0 rows affected (0.13 sec)

INSERTION OF VALUES INTO THE TABLE

```
mysql> insert into student (sn,regno,address)values('shalini',1001,'second street,chennai-73');
Query OK, 1 row affected (0.08 sec)

mysql> insert into student (sn,regno,address)values('vinosha',1002,'kurinji street,chennai-73');
Query OK, 1 row affected (0.06 sec)

mysql> insert into student (sn,regno,address)values('bharani',1003,'First street,chennai-73');
Query OK, 1 row affected (0.04 sec)

mysql> select * from student;
+-----------+---------+--------------------------------+
| sn        | regno | address                        |
+-----------+---------+--------------------------------+
| shalini |  1001 | second street,chennai-73  |
| vinosha |  1002 | kurinji street,chennai-73 |
| bharani |  1003 | First street,chennai-73   |
+-----------+---------+--------------------------------+
3 rows in set (0.00 sec)
```

UPDATING THE TABLE

```
mysql> alter table student add sid integer;
Query OK, 3 rows affected (0.34 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> update student set sid=1 where regno=1001;
Query OK, 1 row affected (0.04 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> update student set sid=2 where regno=1002;
Query OK, 1 row affected (0.03 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> update student set sid=3 where regno=1003;
Query OK, 1 row affected (0.04 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from student;
+-----------+---------+--------------------------------+-------+
| sn        | regno | address                        | sid   |
+-----------+---------+--------------------------------+-------+
| shalini |  1001 | second street,chennai-73  |     1 |
| vinosha |  1002 | kurinji street,chennai-73 |     2 |
| bharani |  1003 | First street,chennai-73   |     3 |
+-----------+---------+--------------------------------+-------+
3 rows in set (0.00 sec)
```

VIEW RECORDS BASED ON CONDITION

```
mysql> select * from student where regno=1001;
+     ------+---  ---+---  ------------------------+---  --+---
| sn      | regno | address                  | sid  |
+     ------+---  ---+---  ------------------------+---  --+---
| shalini |  1001 | second street,chennai-73 |    1 |
+     ------+---  ---+---  ------------------------+---  --+---
1 row in set (0.00 sec)

mysql> select * from student where regno=1001 or sid=2;
+     ------+---  ---+---  ------------------------+---  --+---
| sn      | regno | address                  | sid  |
+     ------+---  ---+---  ------------------------+---  --+---
| shalini |  1001 | second street,chennai-73  |    1 |
| vinosha |  1002 | kurinji street,chennai-73 |    2 |
+     ------+---  ---+---  ------------------------+---  --+---
2 rows in set (0.00 sec)

mysql> select * from student where regno not in(1001,1002);
+     ------+---  ---+---  ------------------------+---  --+---
| sn      | regno | address                  | sid  |
+     ------+---  ---+---  ------------------------+---  --+---
| bharani |  1003 | First street,chennai-73 |    3 |
+     ------+---  ---+---  ------------------------+---  --+---
1 row in set (0.00 sec)
```

DELETION

```
mysql> delete from student where sid=1; Query OK, 1
row affected (0.07 sec)

mysql> select * from student;

+     ------+---  ---+---  ------------------------+---  --+---

| sn      | regno | address                  | sid  |
+     ------+---  ---+---  ------------------------+---  --+---
| vinosha |  1002 | kurinji street,chennai-73 |    2 |
| bharani |  1003 | First street,chennai-73   |    3 |
+     ------+---  ---+---  ------------------------+---  --+---
2 rows in set (0.00 sec)
```

RESULT

Thus, the database was created and MYSQL queries to perform DML commands inserting, deleting, updating and retrieving tables were implemented.

EX.NO:3A IMPLEMENTATION OF DIFFERENT TYPES OF WHERE CLAUSE CONDITIONS

AIM:

To implement different types of where clause conditions in SQL.

COMMANDS:

```
mysql> create database t;
Query OK, 1 row affected (0.02 sec)

mysql> use t;
Database changed

mysql> create table student(rollno integer,sname varchar(20),marks integer);
Query OK, 0 rows affected (0.10 sec)

mysql> show tables;
+-------------+
| Tables_in_t |
+-------------+
| student     |
+-------------+
1 row in set (0.01 sec)

mysql> desc student;
+--------+-------------+------+-----+---------+-------+
| Field  | Type        | Null | Key | Default | Extra |
+--------+-------------+------+-----+---------+-------+
| rollno | int         | YES  |     | NULL    |       |
| sname  | varchar(20) | YES  |     | NULL    |       |
| marks  | int         | YES  |     | NULL    |       |
+--------+-------------+------+-----+---------+-------+
3 rows in set (0.00 sec)

mysql> insert into student(rollno,sname,marks)value(1001,'yamini',95);
Query OK, 1 row affected (0.02 sec)

mysql> select * from student;
+--------+--------+-------+
| rollno | sname  | marks |
+--------+--------+-------+
|   1001 | yamini |    95 |
+--------+--------+-------+
1 row in set (0.00 sec)

mysql> insert into student(rollno,sname,marks)value(1002,'rose',84);
Query OK, 1 row affected (0.01 sec)
mysql> insert into student(rollno,sname,marks)value(1003,'jennie',98);
Query OK, 1 row affected (0.01 sec)
mysql> select * from student;
```

```
+--------+--------+-------+
| rollno | sname  | marks |
+--------+--------+-------+
|   1001 | yamini |    95 |
|   1002 | rose   |    84 |
|   1003 | jennie |    98 |
+--------+--------+-------+

3 rows in set (0.00 sec)

mysql> select sname,marks from student where rollno=1003;
+--------+-------+
| sname  | marks |
+--------+-------+
| jennie |    98 |
+--------+-------+
1 row in set (0.00 sec)

mysql> update student set marks=100 where sname='yamini';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> select * from student;
+--------+--------+-------+
| rollno | sname  | marks |
+--------+--------+-------+
|   1001 | yamini |   100 |
|   1002 | rose   |    84 |
|   1003 | jennie |    98 |
+--------+--------+-------+
3 rows in set (0.00 sec)

mysql> delete from student where rollno=1003;
Query OK, 1 row affected (0.01 sec)

mysql> select * from student;
+--------+--------+-------+
| rollno | sname  | marks |
+--------+--------+-------+
|   1001 | yamini |   100 |
|   1002 | rose   |    84 |
+--------+--------+-------+
2 rows in set (0.00 sec)
```

Where Clause with Logical Operators

```
mysql> select * from student where rollno=1001;
+--------+--------+-------+
| rollno | sname  | marks |
+--------+--------+-------+
|   1001 | yamini |   100 |
+--------+--------+-------+
1 row in set (0.00 sec)
```

Where Clause with BETWEEN Operator

```
mysql> select * from student where marks between 80 and 90;
+--------+--------+-------+
| rollno | sname  | marks |
+--------+--------+-------+
|   1002 | rose   |    84 |
+--------+--------+-------+
1 row in set (0.00 sec)
```

Where Clause with LIKE Operator

```
mysql> select * from student where sname like ‘r%’;

+--------+--------+-------+
| rollno | sname  | marks |
+--------+--------+-------+
|   1002 | rose   |    84 |
+--------+--------+-------+
```

Where Clause with IN Operator

```
mysql> select * from student where marks in(84,100);
+--------+--------+-------+
| rollno | sname  | marks |
+--------+--------+-------+
|   1001 | yamini |   100 |
|   1002 | rose   |    84 |
+--------+--------+-------+
2 rows in set (0.00 sec)
```

Result

Thus the different types of where clause conditions in SQL were implemented.

EX.NO:3B IMPLEMENTATION OF AGGREGATE FUNCTIONS

Aim:

To implement an aggregate function in SQL.

Commands:

```
mysql> create database t;
Query OK, 1 row affected (0.02 sec)

mysql> use t;
Database changed

mysql> create table student(rollno integer,sname varchar(20),marks integer);
Query OK, 0 rows affected (0.10 sec)

mysql> show tables;
+-------------+
| Tables_in_t |
+-------------+
| student     |
+-------------+
1 row in set (0.01 sec)

mysql> desc student;
+--------+-------------+------+-----+---------+-------+
| Field  | Type        | Null | Key | Default | Extra |
+--------+-------------+------+-----+---------+-------+
| rollno | int         | YES  |     | NULL    |       |
| sname  | varchar(20) | YES  |     | NULL    |       |
| marks  | int         | YES  |     | NULL    |       |
+--------+-------------+------+-----+---------+-------+
3 rows in set (0.00 sec)

mysql> insert into student(rollno,sname,marks)value(1001,'yamini',95);
Query OK, 1 row affected (0.02 sec)

mysql> select * from student;
+--------+--------+-------+
| rollno | sname  | marks |
+--------+--------+-------+
|   1001 | yamini |    95 |
+--------+--------+-------+
1 row in set (0.00 sec)

mysql> insert into student(rollno,sname,marks)value(1002,'rose',84);
Query OK, 1 row affected (0.01 sec)

mysql> insert into student(rollno,sname,marks)value(1003,'jennie',98);
Query OK, 1 row affected (0.01 sec)

mysql> select * from student;
+--------+--------+-------+
| rollno | sname  | marks |
+--------+--------+-------+
|   1001 | yamini |    95 |
```

```
|   1002 | rose   |    84 |
|   1003 | jennie |    98 |
+--------+--------+-------+
3 rows in set (0.00 sec)

mysql> select count(marks) from student;
+--------------+
| count(marks) |
+--------------+
|            3 |
+--------------+
1 row in set (0.00 sec)

mysql> select sum(marks) from student;
+------------+
| sum(marks) |
+------------+
|        277 |
+------------+
1 row in set (0.01 sec)

mysql> select avg(marks) from student;
+------------+
| avg(marks) |
+------------+
|    92.3333 |
+------------+
1 row in set (0.00 sec)

mysql> select max(marks) from student;
+------------+
| max(marks) |
+------------+
|         98 |
+------------+
1 row in set (0.00 sec)

mysql> select min(marks) from student;
+------------+
| min(marks) |
+------------+
|         84 |
+------------+
1 row in set (0.00 sec)
```

Result

Thus the aggregate functions in SQL were implemented.

EX.NO:4 IMPLEMENTATION OF DIFFERENT TYPES OF OPERATORS IN SQL

AIM:

To implement different types of operators using SQL.

TYPES OF OPERATORS IN DBMS:

ARITHMETIC OPERATORS:

(+) : Addition - Adds values on either side of the operator .

(-):Subtraction - Subtracts right hand operand from left hand operand .

(*):Multiplication - Multiplies values on either side of the operator .

(/):Division - Divides left hand operand by right hand operand .

(^):Power- raise to power of .

(%):Modulus - Divides left hand operand by right hand operand and returns remainder.

LOGICAL OPERATORS:

AND : The AND operator allows the existence of multiple conditions in an SQL statement's WHERE clause.

OR: The OR operator is used to combine multiple conditions in an SQL statement's WHERE clause.

COMPARISION OPERATORS:

(=): Checks if the values of two operands are equal or not, if yes then condition becomes true. **(!=):**Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.

(< >): Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.

(>): Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.

(<): Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.

(>=): Checks if the value of left operand is greater than or equal to the value of right operand,if yes then condition becomes true.

(<=): Checks if the value of left operand is less than or equal to the value of right operand, ifyes then condition becomes true.

COMMANDS:

Examples for Arithmetic Operators:

Create numbers Table:

```
CREATE TABLE numbers (  num1 INT,    num2 INT );
INSERT INTO numbers (num1, num2)
VALUES (5, 10), (15, 20);
```

1. Addition (+)

The **addition operator** (+) in **MySQL** is used to add numeric values together.

```
SELECT num1, num2, num1 + num2 AS sum FROM numbers;
```

Output:

```
+------+------+------+
| num1 | num2 | sum  |
+------+------+------+
|    5 |   10 |   15 |
|   15 |   20 |   35 |
+------+------+------+
```

2. Subtraction (-)

The **subtraction operator** (-) in MySQL is used to subtract one numeric value from another.

```
SELECT num1, num2, num1 - num2 AS difference FROM numbers;
```

Output:

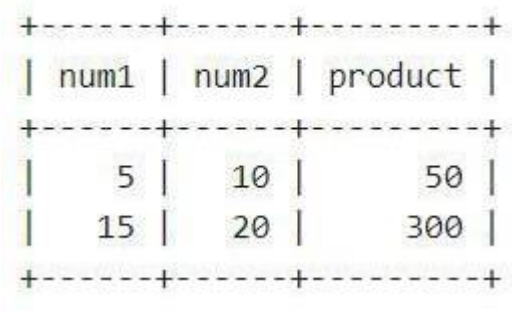
```
+------+------+------------+
| num1 | num2 | difference |
+------+------+------------+
|    5 |   10 |         -5 |
|   15 |   20 |         -5 |
+------+------+------------+
```

3. Multiplication (*)

The multiplication **operator** (*) in MySQL is used to multiply numeric values together.

```
SELECT num1, num2, num1 * num2 AS product FROM numbers;
```

Output:



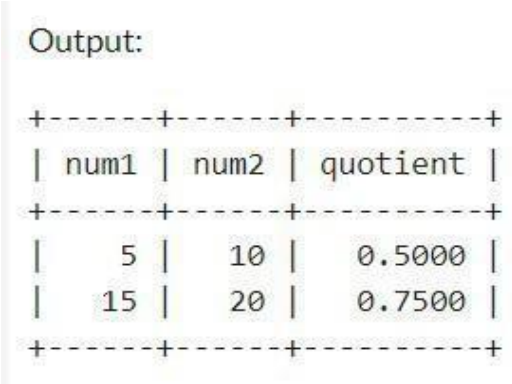
num1	num2	product
5	10	50
15	20	300

4. Division (/)

The **division operator** (/) in MySQL is used to divide one numeric value by another.

SELECT num1, num2, num1 / num2 AS quotient FROM numbers;

Output:



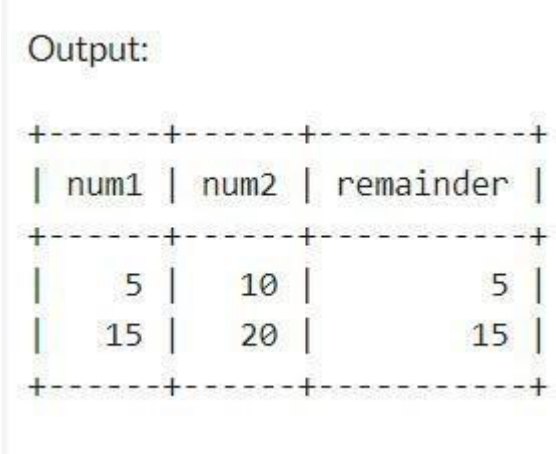
num1	num2	quotient
5	10	0.5000
15	20	0.7500

6. Modulus (%)

The **modulus operator** (%) in MySQL returns the remainder of dividing one numeric value by another.

SELECT num1, num2, num1 % num2 AS remainder FROM numbers;

Output:



num1	num2	remainder
5	10	5
15	20	15

Examples for Logical and Comparison Operators:

SQL>SELECT * FROM TECSE;

ROLL	NAME	DBMS	JAVA
1	KUMBHAR	15	25
2	MALI	40	45
3	SHELKE	35	22
4	THORAT	50	42
5	MORE	45	48

SQL>SELECT ROLL, NAME FROM TECSE WHERE DBMS>30 OR JAVA>35;

ROLL	NAME
1	KUMBHAR
2	MALI
3	SHELKE
4	THORAT
5	MORE

SQL>SELECT ROLL,NAME FROM TECSE WHERE DBMS>30 AND JAVA>32;

ROLL	NAME
2	MALI
4	THORAT

SQL>SELCET ROLL, NAME FROM TECSE WHERE DBMS>30

;ROLL NAME

---------- ----------

2 MALI

3 SHELKE

4 THORAT

RESULT:

Thus, the above SQL queries was successfully executed and verified.

EX. NO. : 5 IMPLEMENTATION OF DIFFERENT TYPES OF JOINS

AIM:

To study about the implementation of different types of joins using SQL.

Joins:

Join queries are used to retrieve data from multiple tables.

TYPES OF JOINS:

EQUI join:

Equi joins are done by equating the tables based on criteria. It uses ,=, symbol.

Cartesian join:

Cartesian join is all combination of rows of different tables.

Non-equi join:

Non-Equi joins are than'=' operator if join is formed using anyone of the relation operators(<,>,!=,<=,>=,<>);

Self join:

It retrieves the row from the table which join with itself.

Outer join:

It retrieves the row returned by EQUI join as well as the row that do not matter any row from tables.outer joins are of two types

1) Right outer join
2) Left outer join

Cross join:

It makes a cartesian product of rows from multiple tables. Suppose, join t1 and t2 tables using the cross join, the result set will include the combinations of rows from the t1 table with the rows in the t2 table.

Inner join:

It requires rows in the two joined tables to have matching column values, the inner join creates the result set by combining column values of two joined tables based on the join- predicate.

To join two tables, the inner join compares each row in the first table with each row in the second table to find pairs of rows that satisfy the join-predicate. whenever the join- predicate is satisfied by matching non-null values, column values for each matched pair of rows of the two tables are included in the result set.

Left join:

It requires a join-predicate when joining two tables using a left join, the concepts of left table and right table are introduced.

A left join returns all rows in the left table including rows that satisfy join-predicate and rows that do not. for the rows that do not match the join-predicate, nulls appear in the columns of the right table in the result set.

Right join:

It is similar to the left join except that the treatment of tables is reversed. with a right join, every row from the right table (t2) will appear in the result set. for the rows in the right table that do not have the matching rows in the left table (t1), nulls appear for columns in the left table (t1).

COMMANDS

mysql>create table t1 (id int primary key,pattern varchar(50) not null); Query OK, 0 rows affected (0.13 sec)

mysql>create table t2 (id varchar(50) primary key,pattern varchar(50) not null); Query OK, 0 rows affected (0.13 sec)

mysql> insert into t1(id, pattern)values(1,'divot'),(2,'brick'),(3,'grid'); Query OK, 1 row affected (0.08 sec)

mysql> insert into t2(id, pattern)values('a','brick'),('b','grid'),('c','diamond'); Query OK, 1 row affected (0.08 sec)

mysql>select t1.id, t2.id from t1 cross join t2;

id	id
1	c
1	b
1	a
2	c
2	b
2	a
3	c
3	b
3	a

9 rows in set (0.00 sec)

mysql>select t1.id, t2.id from t1 inner join t2 on t1.pattern=t2.pattern;

id	id
2	a
3	b

2 rows in set (0.00 sec)

mysql>select t1.id, t2.id from t1 left join t2 on t1.pattern=t2.pattern order by t1.id;

id	id
1	NULL
2	a
3	b

3 rows in set (0.00 sec)

mysql>select t1.id, t2.id from t1 right join t2 on t1.pattern=t2.pattern order by t2.id;

id	id
2	a
3	b
NULL	c

3 rows in set (0.00 sec)

RESULT

Thus the implementation of various join operations was executed successfully.

Ex. No. 6 STUDY AND IMPLEMENTATION OF SUB QUERIES IN SQL

AIM:

To study and implement sub queries in SQL.

1. Given Application:

Given relational schema are

- sempls(enoprimarykey,ename,edob,gender,doj,desg,bpay,dno)
- sdments (dno, dname)
- spros(pno primarykey,pname,dno)
- sworks(eno, pno, datework,intime, outtime)

A department can control any number of projects but a project is controlled only by one department. An employee can work in any number of projects on a day but an employee isnot permitted to work morethan onceon aproject on the same day. Develop suitablequeries.

*********USAGE OF COMMANDS WITH EXAMPLES:*********

2.TOCREATE'SEMPLS'TABLE

```
mysql> create table sempls( eno integerprimary key, ename varchar(20), edob varchar(15), gender
    varchar(10), doj varchar(15),desg varchar(30), bpay number(10), dno number(10)); Query OK, 0 rows
    affected (0.00 sec)
mysql>insertintosemplsvalues(1,'bala','15/1/84','m','16thjuly', lec',7000,1);
Query OK, 0 rows affected (0.13 sec)
mysql>insertintosemplsvalues(2,'kala','13/9/84','m','18thjuly',' lec',10000,2);
Query OK, 0 rows affected (0.13 sec)

mysql>insertintosemplsvalues(3,'mala','17/6/85','f','19thjune',' lec',19000,1);
Query OK, 0 rows affected (0.13 sec)

mysql>insertintosemplsvalues(4,'nila','20/6/85','f','19thjune','s r.lec',20000,1);
Query OK, 0 rows affected (0.13 sec)

mysql>insertintosemplsvalues(5,'vina','2/1/85','f','12thjuly','pr of.',50000,2);
Query OK, 0 rows affected (0.13 sec)
   mysql>select*fromsempls;
```

eno	ename	edob	gender	doj	desg	bpay	dno
1	bala	15/1/84	m	16thJuly	lec	7000	1
2	kala	13/9/84	m	18thJuly	lec	10000	2
3	mala	17/6/85	f	19thJune	lec	19000	1
4	nila	20/6/85	f	19thJune	sr.lec	20000	1
5	vina	2/1/85	f	12th july	prof.	50000	2

+ ------ +----------+----------+----------+----------+--------.+---------+-------+

5rows in set (0.00 sec)

3. TOCREATE‘SDMENTS’TABLE

mysql>createtablesdments(dnointeger(10),dnamevarc har(30)); Query OK, 0 rows affected (0.00 sec)

mysql>insertintosdmentsvalues(1,'c se'); Query OK, 0 rows affected (0.08 sec)

mysql>insertintosdmentsvalues(2,'it'); Query OK, 0 rows affected (0.08 sec)

mysql>select*fromsdments;

+--------+---------+
|DNO|DNAME|
+--------+---------+
|1 |cse |
|2 | it |
+--------+---------+
2rowsin set(0.00 sec)

3. TOCREATE‘SPROS’TABLE

mysql>createtablespros(pno integer(20)
primarykey,pname varchar2(30),dno integer(10));
QueryOK,0rowsaffected(0.00 sec)

mysql>insertintosprosvalues(81,'aaa ',1); Query OK, 0 rows affected
(0.08 sec)

mysql>insertintosprosvalues(82,'bb b',1); Query OK, 0 rows affected (0.08 sec)

mysql>insertintosprosvalues(83,'ccc ',1); Query OK, 0 rows affected
(0.08 sec)
mysql>insertintosprosvalues(84,'dd d',2); Query OK, 0 rows affected (0.08 sec)

mysql>insertintosprosvalues(85,'eee ',2);

Query OK, 0 rows affected
(0.08 sec)

mysql>select*fromspros;

PNO	PNAME	DNO
81	aaa	1
82	bbb	1
83	ccc	1
84	ddd	2
85	eee	2

5rowsin set(0.00 sec)

3. TOCREATE‘SWORKS’TABLE

mysql> create table sworks (eno integer(10) , pno integer(20) , datework varchar(20) , intime integer(10),outtime integer(10)); QueryOK,0rowsaffected(0.00 sec)

mysql>insertintosworksvalues(1,81,'11th july',9,10); Query OK, 0
rows affected (0.08 sec)

mysql>insertintosworksvalues(1,82,'11thjuly', 10,11); Query OK, 0 rows affected
(0.08 sec)

mysql>insertintosworksvalues(1,83,'11thjuly', 11,12); Query OK, 0 rows affected
(0.08 sec)

mysql>insertintosworksvalues(1,84,'11th july',12,1); Query OK, 0
rows affected (0.08 sec)

mysql>insertintosworksvalues(1,85,'11thjuly',1,2); Query OK, 0 rows affected (0.08 sec)

mysql>insertintosworksvalues(2,85,'12th july',8,9); Query OK, 0
rows affected (0.08 sec)

mysql>select*fromsworks;

ENO	PNO	DATEWORK	INTIME	OUTTIME
1	81	11th july	9	10
1	82	11th july	10	11
1	83	11th july	11	12
1	84	11th july	12	1

1	85	11th july	1	2
2	85	12th july	8	9

6rowsin set(0.00 sec)

QUERY1

Thisquerylistsdetailsofemployeeswhoearnabasicpaythatislessthanth eaverage basic pay of the employees.

mysql>select*from semplswherebpay<(selectavg(bpay)from sempls);

eno	ename	edob	gender	doj	desg	bpay	dno
1	bala	15/1/84	m	16thJuly	lec	7000	1
2	kala	13/9/84	m	18thJuly	lec	10000	2
3	mala	17/6/85	f	19thJune	lec	19000	1
4	nila	20/6/85	f	19thJune	lec	20000	1

4rowsin set(0.00 sec)

QUERY2

Thisquerylistsdepartmentnumber,numberofemployeesineachd epartment**.** mysql> selectdno,count(eno) from sempls group by dno;

DNO	COUNT(ENO)
1	3
2	2

2rowsin set(0.00 sec)

QUERY3

Thisquerylistsdetailsofemployeeswhoearnabasicpayintherange1000 0to20000. mysql> select * from sempls where bpay between 10000 and 20000;

eno	ename	edob	gender	doj	desg	bpay	dno
2	kala	13/9/84	m	18thJuly	lec	10000	2
3	mala	17/6/85	f	19thJune	lec	19000	1
4	nila	20/6/85	f	19thJune	lec	20000	1

3rowsin set(0.00 sec)

QUERY4

Thisquerylistsdetailsofemployeeswhohaveworkedinprojectscontrol ledby department name = cse.

```
mysql>select * from sempls,sdments,spros where
        sdments.dno=spros.dno and
sdments.dno=sempls.dno and dname='cse';
+------+--------+-----------+---------+----------+------+---------+------+-------+---------+-----+--------+-----+
|eno|ename|edob |gender|doj
desg|bpay|dno|dno|dname|pno|pname|dno|
------ --------- ---------- ---------- --------- ------ ---------- ------ ------- ---------- ----- --------- -----
|1  |bala|15/1/84|m  |16thJuly|lec |7000|1|1  |cse  |81|aaa  |1  |
|3  |mala|17/6/85|f  |19thJune| lec |19000|1|1  |cse  |81|aaa  |1  |
|4  |nila|20/6/85|f  |19thJune|sr.lec|20000|1|1  |cse  |81|aaa  |1  |
+ ------+--------+-----------+---------+----------+------+---------+------+-------+---------+----+--------+----+
|3rowsin set(0.00 sec)
```

QUERY5

This query lists the employee number, employeename, department number, date worked if the employee has worked in more than 4 projects on a day.

```
mysql> select sempls.eno,ename,dno,datework from sempls,sworks where sempls.eno in(select eno
from (select eno,datework from sworks group by eno,datework having count(pno)>4))and datework in
(select datework from (select eno,datework from sworks group by eno,datework ha
ving count(pno)>4));

+ -----+---- -----+------ +--------- ------------------------+
 |ENO|ENAME|DNO|              DATEWORK|
+ -------+-- --------+- ------------+- ------------------------+-
|      1|bala       |1           |   11th july          |
|      1|bala       |1           |   11th july          |
|      1|bala       |1           |   11th july          |
|      1|bala       |1           |   11th july          |
+ -------+-- ----------+- --------+- ------------------------+-

6rowsin set(0.00 sec)
```

RESULT

Thus the study and implementation of sub queries in SQL was successfully executed.

EX. NO.: 7 STUDY AND IMPLEMENTATION OF PATTERN MATCHING IN SQL

AIM:

To study and implementation of pattern matching in sql.

PROCEDURE:

Pattern matching in mysql primarily revolves around two key operators: **LIKE** and **REGEXP**. Such operators allow text sorting within text field types like strings and characters.

1. LIKE Operator

This **LIKE operator** may be the one most often used in MySQL queries to run pattern matching. It finds an exact match to the given pattern inside the string. Pattern structure will comprise wildcards to represent any number of symbols or a single symbol.

Syntax: **SELECT * FROM products WHERE product_name LIKE 'S%';**

- **'%' Wildcard**: The percent sign '%' denotes zero or more characters. As an example, '%apple' obtains any string that begins with '%apple', followed by zero or more characters.
- **" Wildcard**: Underscore " is used to show a character's presence. For instance, 'c_t' corresponds to 'cat, cot, cut', etc.
- **Combination**: Adding '%' and '_' together allows for a more complicated form. For example, 'a%e' matches any string that began with 'a' and ended with 'e'.

2. REGEXP Operator

The REGEXP operator gives a more complex matching option by utilizing regular expressions for matching patterns. Regular expressions are powerful pattern-matching tools that allow for sophisticated searches involving complicated and specific patterns within text data. Syntax:

SELECT * FROM products WHERE product_name REGEXP '[0-9]';

- **Character Classes**: Regular expressions will include character classes such as [a-z] for any lowercase letter and [0-9] for any digit.
- **Quantifiers**: Quantifiers specify the number of occurrences of a character or group, such as '*' for zero or more occurrences and '+' for one or more occurrences.
- **Anchors**: Anchors specify the position of a pattern within a string, such as '^' for the start of a string and '$' for the end of a string.

COMMANDS:

Creation of table:

mysql>CREATE TABLE products (product_id INT AUTO_INCREMENT PRIMARY KEY, product name VARCHAR(100));

mysql >INSERT INTO products (product_name) VALUES
('Apple iPhone 12'),

('Samsung Galaxy S20'),
('Google Pixel 5'),
('Huawei P40 Pro'),
('OnePlus 8T');

mysql>Select * from products;

product_id	product_name
1	Apple iPhone 12
2	Samsung Galaxy S20
3	Google Pixel 5
4	Huawei P40 Pro
5	OnePlus 8T

Using the LIKE Operator:

mysql>SELECT * FROM products WHERE product_name LIKE 'S%';

product_id	product_name
2	Samsung Galaxy S20

Using the REGEXP Operator:

mysql>SELECT * FROM products WHERE product_name REGEXP '[0-9]';

product_id	product_name
1	Apple iPhone 12
2	Samsung Galaxy S20
5	OnePlus 8T

Using LIKE with '%' and '_':

mysql>SELECT * FROM products WHERE product_name LIKE 'A%e';

product_id	**product_name**
1	**Apple iPhone 12**
5	**OnePlus 8T**

Result:

Hence to study and implementation of pattern matching is done successfully.

EX. NO.:8 STUDY AND IMPLEMENTATION OF DIFFERENT TYPE OF CONSTRAINTS

AIM:

To practice and implement different type of constraints like primary key, not null, default, check, unique**.**

PROCEDURE:
CONSTRAINTS:

Constraints are used to specify rules for the data in a table. If there is any violation betweenthe constraint and the data action, the action is aborted by the constraint. It can be specifiedwhen the table is created (using CREATE TABLE statement) or after the table is created (using ALTER TABLE statement).

1. **NOT NULL:** When a column is defined as NOTNULL, then that column becomes a mandatory column. It implies that a value must be entered into the column if the record is to be accepted for storage in the table.

2. **UNIQUE:** The purpose of a unique key is to ensure that information in the column(s) is unique i.e. a value entered in column(s) defined in the unique constraint must not be repeated across the column(s). A table may have many unique keys.

3. **CHECK:** Specifies a condition that each row in the table must satisfy. To satisfy the constraint, each row in the table must make the condition either TRUE or unknown (due to a null).

4. **PRIMARY KEY:** A field which is used to identify a record uniquely. A column or combination of columns can be created as primary key, which can be used as a reference from other tables. A table contains primary key is known as Master Table.
It must uniquely identify each record in a table.It must contain unique values.
It cannot be a null field.
It cannot be multi port field.
It should contain a minimum no. of fields necessary to be called unique.

5. **FOREIGN KEY:** It is a table level constraint. We cannot add this at column level. To reference any primary key column from other table this constraint can be used. The table in which the foreign key is defined is called a **detail table**. The table that defines the primary key and is referenced by the foreign key is called the **master table**.

SQL>CREATE TABLE CSE1(ROLL INT PRIMARY KEY,NAME VARCHAR(20),CITY VARCHAR(20));

TABLE CREATED.

SQL>DESC CSE1

NAME	NULL?	TYPE
ROLL	NOT NULL	NUMBER(38)
NAME		VARCHAR(20)
CITY		VARCHAR(20)

SQL>SQL>ED

Wrote file afiedt.buf

1* CREATE TABLAE CSE2(ROLL INT PRIMARY KEY,NAME VARCHAR(20) NOT NULL, CITY VARCHAR(20))

SQL>/

TABLE CREATED

SQL>DESC CSE2

NAME	NULL	TYPE
ROLL	NOT NULL	NUMBER(38)
NAME	NOT NULL	VARCHAR(20)
CITY		VARCHAR(20)

SQL>ED

Wrote file afiedt.buf

1* CREATE TABLAE STUDENT(ROLL INT UNIQUE, NAME VARCHAR(20) , CITY VARCHAR(20))

SQL>/

TABLE CREATED

SQL>DESC STUDENT

NAME	NULL	TYPE
ROLL	NOT	NUMBER (38)

NAME

VARCH

AR(20) CITY

VARCH

AR(20)

1* CREATE TABLAE STUDENT1(ROLL INT UNIQUE ,NAME VARCHAR(20) , CITYVARCHAR(20),MARK INT DEFAULT 40)

TABLE

CREATED

SQL>DES

C

STUDENT

NAME	NULL	TYPE
ROLL	NOT	NUMBER(38)

NAME

VARCH

AR(20) CITY

VARCH

AR(20) SQL>ED

Wrote file afiedt.buf

1* CREATE TABLAE STUDENT1 (ROLL INT UNIQUE,NAME VARCHAR(20), CITYVARCHAR(20),MARK INT DEFAULT 40)

SQL>/

TABLE CREATED

SQL>DESC STUDENT1

NAME	NULL	TYPE
ROLL	NOT	NUMBER(38)
NAME		VARCHAR(20)
CITY		VARCHAR(20)

SQL>INSERT INTO STUDENT1 VALUES(1,'PRAGATI','AUSA',84);

1 row created.
SQL>
ED

Wrote file afiedt.buf

1* INSERT INTO STUDENT1

VALUES(2,'SWARUPA','LOHARA',64); SQL>/

INSERT INTO STUDENT1 VALUES(2,'SWARUPA','LOHARA',64);

ERROR AT LINE 1

ORA-00947:not enough values

RESULT:

Thus, the above SQL queries was successfully executed and verified.

Ex. No. 9 **USER DEFINED FUNCTIONS IN SQL**

Aim:

To write user defined functions in SQL.

Scalar User Defined Function

A scalar function returns a single value (e.g., a number or string).

```
CREATE FUNCTION dbo.SquareNumber (@Number INT)
RETURNS INT
AS
BEGIN
   RETURN @Number * @Number
END
```

Usage:

```
SELECT dbo.SquareNumber(5) AS Result  -- Returns 25
```

Table-Valued User Defined Function (Inline)

This type returns a **table** and can be used like a view or join

Example: Get all employees with salary above a given value

```
CREATE FUNCTION dbo.GetHighPaidEmployees (@MinSalary DECIMAL(10,2))

RETURNS TABLE

AS

RETURN

(

   SELECT EmployeeID, Name, Salary

   FROM Employees

   WHERE Salary > @MinSalary

)
```

Usage:

```
SELECT * FROM dbo.GetHighPaidEmployees(50000)
```

Multi-Statement Table-Valued Function

If you need more logic (e.g., variables, loops), use this version.

Example: Return a table of numbers and their squares up to a given limit

```
CREATE FUNCTION dbo.GenerateSquares (@MaxNumber INT)

RETURNS @Result TABLE (Number INT, Square INT)

AS

BEGIN
```

```
    DECLARE @i INT = 1
    WHILE @i <= @MaxNumber
    BEGIN
        INSERT INTO @Result (Number, Square)
        VALUES (@i, @i * @i)

        SET @i = @i + 1
    END
    RETURN
END
```

Usage:

```
SELECT * FROM dbo.GenerateSquares(5)
```

Returns:

Number	Square
1	1
2	4
3	9
4	16
5	25

Result:

Thus the user defined functions in SQL was executed successfully

Ex.No:10 **STORED PROCEDURES IN SQL**

AIM:

To demonstrate how to create and use a stored procedure in MySQL that accepts input parameters, returns output parameters, and modifies input/output parameters within the procedure.

Definition:

A **Stored Procedure** is a set of SQL statements that you save and reuse later. It allows you to encapsulate logic inside the database, making it reusable, easier to maintain, and sometimes faster.

Basic Syntax of a Stored Procedure (in SQL Server):

```
CREATE PROCEDURE procedure_name
   @parameter1 datatype,
   @parameter2 datatype OUTPUT -- Optional output parameter
AS
BEGIN
   -- SQL statements here
   SELECT * FROM table_name WHERE column_name = @parameter1;

   -- You can have complex logic, control flow, etc.
END;
```

Example:

Step 1: Create Sample Table

```
mysql> create database db;
Query OK, 1 row affected (0.02 sec)
mysql> use db;
Database changed
CREATE TABLE Employees (
   EmployeeID INT PRIMARY KEY,
   FirstName VARCHAR(50),
   LastName VARCHAR(50),
   Department VARCHAR(50),
   Salary DECIMAL(10,2)
);

Query OK, 0 rows affected (0.10 sec)
```

Step 2: Insert Sample Data

```
INSERT INTO Employees VALUES
(1, 'John', 'Doe', 'HR', 50000),
(2, 'Jane', 'Smith', 'IT', 60000),
(3, 'Alice', 'Johnson', 'Finance', 70000);
```

```
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

Stored Procedure with IN **Parameter**

Purpose:

Take an employee ID as input and return the employee's details.

```
DELIMITER //

CREATE PROCEDURE GetEmployeeByID(IN emp_id INT)
BEGIN
   SELECT * FROM Employees WHERE EmployeeID = emp_id;
END //

Query OK, 0 rows affected (0.05 sec)
DELIMITER ;
CALL GetEmployeeByID(2);

+------------+-----------+----------+------------+----------+
| EmployeeID | FirstName | LastName | Department | Salary   |
+------------+-----------+----------+------------+----------+
|          2 | Jane      | Smith    | IT         | 60000.00 |
+------------+-----------+----------+------------+----------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

Stored Procedure with OUT **Parameter**

Purpose:

Return the salary of an employee using an OUT parameter.

```
DELIMITER //

CREATE PROCEDURE GetEmployeeSalary(
   IN emp_id INT,
   OUT emp_salary DECIMAL(10,2)
)
BEGIN
   SELECT Salary INTO emp_salary FROM Employees WHERE EmployeeID = emp_id;
END //

Query OK, 0 rows affected (0.03 sec)
DELIMITER ;
-- Declare a variable to hold output
SET @salary = 0;

Query OK, 0 rows affected (0.00 sec)
```

```
-- Call the procedure
CALL GetEmployeeSalary(2, @salary);

Query OK, 1 row affected (0.00 sec)
-- Check the output
SELECT @salary;
+----------+
| @salary  |
+----------+
| 60000.00 |
+----------+
1 row in set (0.00 sec)
```

Stored Procedure with INOUT **Parameter**

Purpose:

Take a bonus amount as input, add a fixed increment to it inside the procedure, and return the updated bonus.

```
DELIMITER //

CREATE PROCEDURE UpdateBonus(
   INOUT bonus_amount DECIMAL(10,2)
)
BEGIN
   -- Add 500 to the incoming bonus amount
   SET bonus_amount = bonus_amount + 500;
END //

Query OK, 0 rows affected (0.02 sec)

DELIMITER ;
-- Declare a variable with initial bonus
SET @bonus = 1500;
Query OK, 0 rows affected (0.00 sec)

-- Call the procedure with INOUT parameter
CALL UpdateBonus(@bonus);
Query OK, 0 rows affected (0.01 sec)

-- Check the updated bonus value
SELECT @bonus AS UpdatedBonus;
+--------------+
| UpdatedBonus |
+--------------+
|    2000.00 |
+--------------+
1 row in set (0.00 sec)
```

RESULT

Thus the Stored procedures was created and executed with parameters in SQL successfully.

Ex.No. 11a Execute Complex transactions and realize DCL commands

Aim:

To execute complex transactions and realize DCL commands.

DCL (Data Control Language):

It includes commands such as GRANT and REVOKE which mainly deal with the rights, permissions and other controls of the database system. These commands are used to control access to data in the database by granting or revoking permissions.

GRANT:

Assigns new privileges to a user account, allowing access to specific database objects, actions or functions.

Syntax:

GRANT privilege_type [(column_list)] ON [object_type] object_name TO user [WITH GRANT OPTION];

REVOKE:

Removes previously granted privileges from a user account, taking away their access to certain database objects or actions.

Syntax:

REVOKE [GRANT OPTION FOR] privilege_type [(column_list)] ON [object_type] object_name FROM user [CASCADE];

Step 1: Create Database and Table CREATE DATABASE company_db; USE company_db;

CREATE TABLE employees (id INT PRIMARY KEY, name VARCHAR(100),
department VARCHAR(100));

Output:

Query OK, 1 row affected (0.01 sec) Query OK, 0 rows affected (0.00 sec)

Step 2: Create User

CREATE USER 'john'@'localhost' IDENTIFIED BY 'password123';

Output:

Query OK, 0 rows affected (0.01 sec)

Step 3: GRANT Permissions

GRANT SELECT, INSERT ON company_db.employees TO 'john'@'localhost';

Output:

GRANT SELECT, INSERT ON company_db.employees TO 'john'@'localhost';

Step 4: Test as User john

SELECT * FROM company_db.employees;

Output:

Empty set (0.00 sec)

INSERT INTO company_db.employees (id, name, department) VALUES (1, 'Alice', 'HR');

Output:

Query OK, 1 row affected (0.01 sec)

Step 5: REVOKE Permission

REVOKE INSERT ON company_db.employees FROM 'john'@'localhost';

Output:

Query OK, 0 rows affected (0.00 sec)

Step 6: Test REVOKED Access

INSERT INTO company_db.employees (id, name, department) VALUES (2, 'Bob', 'IT');

Output:

ERROR 1142 (42000): INSERT command denied to user 'john'@'localhost' for table 'employees'

SELECT * FROM company_db.employees;

Output:

```
+----+-------++
| id | name | department |
+----+-------++
| 1 | Alice | HR         |
+----+-------++
1 row in set (0.00 sec)
```

Result:

Thus the DCL commands was executed successfully.

Ex. No. 11b **Execute Complex transactions and realize TCL commands**

Aim:

To execute complex transactions and realize TCL commands.

DEFINITION

Transaction Control Language(TCL) commands are used to manage transactions in the database.

COMMIT

This command is used to permanently save any transaction into the database. When we use any DML command like INSERT, UPDATE or DELETE, the changes made by these commands are not permanent, until the current session is closed, the changes made by these commands can be rolled back.To avoid that, we use the COMMIT command to mark the changes as permanent.

Syntax:

COMMIT;

ROLLBACK

This command restores the database to last committed state. It is also used with SAVEPOINT command to jump to a save point in an ongoing transaction.

If we have used the UPDATE command to make some changes into the database, and realise that those changes were not required, then we can use the ROLLBACK command to rollback those changes, if they were not committed using the COMMIT command.

Syntax:

ROLLBACK TO savepoint_name;

SAVEPOINT

This command is used to temporarily save a transaction so that you can rollback to that point whenever required.

Syntax:

SAVEPOINT savepoint_name;

COMMANDS

mysql> create table emp(ename varchar(20), eid integer, salary integer, exp integer);
Query OK, 0 rows affected (0.13 sec)

INSERTION OF VALUES INTO THE TABLE

mysql> insert into emp values(kavi,1,40000,7);
Query OK, 1 row affected (0.08 sec)
mysql> insert into emp values(kar,2,30000,6);
Query OK, 1 row affected (0.06 sec)

mysql> insert into emp values(shanthi,3,40000,9);
Query OK, 1 row affected (0.04 sec)

mysql>start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from emp;

```
+      +        +        +        +
| ename  | eid | salary | exp |
+      +        +        +        +
| kavi |         1 | 40000 |       7 |
| kar  |         2 | 30000 |       6 |
| shanthi |      3 | 40000 |       9 |
+      +        +        +        +
3 rows in set (0.00 sec)
mysql> savepoint ee;
Query OK, 0 rows affected (0.00 sec)
mysql> insert into emp values('uma',6,70000,15); Query OK, 1 row affected (0.01 sec)
mysql> rollback work to savepoint ee; Query OK, 0 rows affected (0.00 sec)

mysql> select * from emp;
+      +        +        +        +
| ename  | eid | salary | exp |
+      +        +        +        +
| kavi |         1 | 40000 |       7 |
| kar  |         2 | 30000 |       6 |
| shanthi |      3 | 40000 |       9 |
+      +        +        +        +
3 rows in set (0.00 sec)

mysql> commit;
Query OK, 0 rows affected (0.05 sec)
```

RESULT

Thus, the TCL commands was executed successfully.

EX. NO: 12 **TRIGGERS**

(Insert, Delete and Update operations in database table)

AIM

To study and implement the concept of triggers.

DEFINITION

A trigger is a statement that is executed automatically by the system as a side effect of a modification to the database. The parts of a trigger are,

• Trigger statement: Specifies the DML statements and fires the trigger body. It also specifies the table to which the trigger is associated.

• Trigger body or trigger action: It is a PL/SQL block that is executed when the triggering statement is used.

• Trigger restriction: Restrictions on the trigger can be achieved

The different uses of triggers are as follows,

• To generate data automatically

• To enforce complex integrity constraints

• To customize complex securing authorizations

• To maintain the replicate table

• To audit data modifications

TYPES OF TRIGGERS

The various types of triggers are as follows, MySQL Triggers are of 4 types:

• Before Insert Trigger

• Before Update Trigger

• After Insert Trigger

• After Update Trigger

VARIABLES USED IN TRIGGERS

• new

• old

These two variables retain the new and old values of the column updated in the database. The values in these variables can be used in the database triggers for data manipulation.

SYNTAX

create or replace trigger trigger_name [before/after] {DML statements} on [table_name] [for each row/statement]

begin

exception end;

USER DEFINED ERROR MESSAGE

The package “raise_application_error” is used to issue the user defined error messages

Syntax: raise_application_error(error number,‘error message‘); The error number can lie between -20000 and -20999.

The error message should be a character string.

BEFORE INSERT

MySQL Before insert trigger will fire the trigger before performing insert records values inside the table.

Syntax:

create trigger trigger_name before insert on <table_name> for each row BEGIN

variable declarations trigger code

END;

trigger_name => The name of the trigger to be created. table_name => The accurate table in the database.

BEFORE UPDATE

MySQL Before update trigger will fired the trigger before performing update records values inside the table.

Syntax

create trigger <trigger_name> before update on <table_name> for each row BEGIN

— variable declarations

— trigger code END;

trigger_name => The name of the trigger to be create. table_name => The accurate table in the database.

AFTER INSERT

MySQL After Insert trigger will fire the trigger after performing insert records values inside the table.

Syntax

create trigger <trigger_name> after insert on <table_name> for each row BEGIN

— variable declarations

— trigger code END;

trigger_name =>The name of the trigger to be create. table_name => The accurate table in the database.

AFTER UPDATE

MySQL Before update trigger will fired the trigger after performing insert records values inside the table.

Syntax

create trigger <trigger_name> after update on <table_name> for each row BEGIN

— variable declarations

— trigger code

END;

trigger_name => The name of the trigger to be create. table_name => The accurate table in the database.

COMMAND

mysql> create table employee(emp_id integer, emp_name varchar(20),city varchar(20), designation varchar(20), salary integer, perks integer);

Query OK, 0 rows affected (0.13 sec)

mysql> insert into employee values(1,’david','delhi','manager',12000,855); Query OK, 1 row affected (0.10 sec)

mysql> insert into employee values(2,'shaha','mumbai','assistant manager',13000,853); Query OK, 1 row affected (0.10 sec)

mysql> insert into employee values(3,’sha','pune','sales manager',11000,850); Query OK, 1 row affected (0.10 sec)

mysql> insert into employee values(4,'jack',’bangalore','designer',14000,854); Query OK, 1 row affected (0.10 sec)

mysql> insert into employee values(5,'james',’mangalore','web designer',15000,1124); Query OK, 1 row affected (0.10 sec)

mysql> select * from employee;

++ + + + + +

| emp_id | emp_name | city | designation | salary | perks |

++ + + + + +

| 1 | david | delhi | manager | 12000 | 855 |

| 2 | shaha | mumbai | assistant manager | 13000 | 853 |

| 3 | sha | pune | sales manager | 11000 | 850 |

| 4 | jack | bangalore | designer | 14000 | 854 |

| 5 | james | mangalore | web designer | 15000 | 1124 |

++ + + + + + 5 rows in set (0.00 sec)

mysql delimiter $$

mysql CREATE TRIGGER inst_trigger BEFORE INSERT ON Employee

-> FOR EACH ROW

-> BEGIN

-> UPDATE Employee SET Salary=Salary-400 where perks>500;

-> end;

-> $$

Query OK, 0 rows affected (0.42 sec)

mysql> delimiter ;

mysql> insert into employee values(6,'mike','chennai','develpoer',15500,840); Query OK, 1 row affected (0.10 sec)

mysql> select * from employee;

++ + + + + +

| emp_id | emp_name | city | designation | salary | perks |

++ + + + + +

| 1 | david | delhi | manager | 11600 | 855 |

| 2 | shaha | mumbai | assistant manager | 12600 | 853 |

| 3 | sha | pune | sales manager | 10600 | 850 |

| 4 | jack | bangalore | designer | 13600 | 854 |

| 5 | james | mangalore | web designer | 14600 | 1124 |

| 6 | mike | chennai | developer | 15100 | 840 |

++ + + + + + 6 rows in set (0.00 sec)

```
mysql> delimiter //
mysql> CREATE TRIGGER updtrigger BEFORE UPDATE ON Employee
-> FOR EACH ROW
-> BEGIN
-> IF NEW.Salary=500 THEN -> SET NEW.Salary=10000;
-> ELSEIF NEW.Salary>500 THEN
-> SET NEW.Salary=15000;
-> END IF;
-> END
-> //
Query OK, 0 rows affected (0.01 sec) mysql> delimiter ;
mysql> UPDATE Employee SET Salary=500; Query OK, 5 rows affected (0.04 sec)
Rows matched: 7 Changed: 5 Warnings: 0

mysql> select * from employee;
++ + + + + +
| emp_id | emp_name | city | designation | salary | perks |
++ + + + + +
| 1 | david | delhi | manager | 10000 | 855 |
| 2 | shaha | mumbai | assistant manager | 10000 | 853 |
| 3 | sha | puna | scales manager | 10000 | 850 |
| 4 | jack | bangalore | designer | 10000 | 854 |
| 5 | james | mangalore | web designer | 10000 | 1124 |
| 6 | mike | chennai | develpoer | 10500 | 840 |
++ + + + + + 6 rows in set (0.00 sec)
```

mysql >create table student_table(stu_id int,stu_name varchar(255),stu_class int); Query OK, 0 rows affected (0.38 sec)

mysql> create table student_log1(user_id varchar(255),description varchar(255)); Query OK, 0 rows affected (0.48 sec)

mysql > CREATE TRIGGER student_insert AFTER insert ON student_table FOR EACH ROW

BEGIN

INSERT into student_log1(user_id,description) VALUES (user(),CONCAT('INSER T student records',new.stu_id,' ',new.stu_name,' ',new.stu_class));

END$$

Query OK, 0 rows affected (0.15 sec) mysql >delimiter ;

mysql >insert into student_table values(1,'mike',10); Query OK, 1 row affected (0.10 sec)

mysql> insert into student_table values(2,'mad',20); Query OK, 1 row affected (0.11 sec)

mysql >insert into student_table values(3,'mack',30); Query OK, 1 row affected (0.10 sec)

mysql> select * from student_log1;

++ +

| user_id | description |

++ +

| root@localhost | INSERT student records1 mike 10 |

| root@localhost | INSERT student records2 mad 20 |

| root@localhost | INSERT student records3 mack 30 |

++ + 3 rows in set (0.00 sec)

mysql>create table Student_Table02(Stu_Id int, Stu_Name varchar(255),Stu_Class int); Query OK, 0 rows affected (0.38 sec)

```
mysql>create table stu_log02( user_id VARCHAR(255), description VARCHAR(255));
Query OK, 0 rows affected (0.38 sec)

mysql>delimiter $$

mysql>CREATE TRIGGER stu_update AFTER UPDATE ON stu_table FOR EACH ROW
->BEGIN
->INSERT into stu_log(user_id, description)
-> VALUES (user(), CONCAT('Update Student Record
->(',old.stu_id,' ',old.stu_name,' ',old.stu_class,
') to (',new.stu_id,' ',new.stu_name,' ',new.stu_class,')'));
->END$$

mysql>delimiter ;
mysql> select * from student_table02;
++        +        +
| stu_id | stu_name | stu_class |
++        +        +
| 1 | david        |         9 |

| 2 | shah         |         9 |
| 3 | mike         |         9 |
| 4 | james        |         9 |
++        +        + 4 rows in set (0.00 sec)

mysql> update student_table02 set stu_class=stu_class+1; Query OK, 4 rows affected (0.20
sec)
```

Rows matched: 4 Changed: 4 Warnings: 0

mysql> select * from student_table02;

++ + +

| stu_id | stu_name | stu_class |

++ + +

| 1 | david | 10 |

| 2 | shah | 10 |

| 3 | mike | 10 |

| 4 | james | 10 |

++ + + 4 rows in set (0.00 sec)

mysql>select * from stu_log02;

++ +

| user_id | description |

++ +

| root@localhost | Update Student Record (1 david 9) to (1 david 10) |

| root@localhost | Update Student Record (2 shah 9) to (2 shah 10) |

| root@localhost | Update Student Record (3 mike 9) to (3 mike 10) |

| root@localhost | Update Student Record (4 james 9) to (4 james 10) |

++ +

RESULT

Thus, the triggers were created and executed.